

Industrial Robot Copilot - Committee Runbook

Project Summary

Industrial Robot Copilot is a multimodal safety and navigation assistant for industrial robots. It combines RGB, thermal, radar, VLM scene understanding, open-vocabulary object localization, risk reasoning, and robot path planning in a Gradio dashboard.

The demo shows a robot-perspective view with:

- detected humans and industrial objects
- risk level and action recommendation
- desired ground-plane path
- speed-coded path segments
- stop point behavior when the robot must halt
- Qwen or heuristic VLM scene reasoning
- locate-anything.cpp bounding boxes

Main Demo Order

Run in this order on the AMD cloud machine:

1. Prepare the Qwen/vLLM environment.
2. Start Qwen VLM server.
3. Start locate-anything.cpp HTTP wrapper.
4. Run benchmark.
5. Build or resume the cached stream output.
6. Start the Gradio demo server.

Terminal 1 - Qwen Environment And Server

From the project root:

```
cd /workspace/AMD_Hackathon
./scripts/fix_aml_vllm_env.sh
./scripts/start_qwen_vllm.sh
```

Expected Qwen endpoint:

```
http://localhost:8000/v1
```

Health check:

```
curl http://localhost:8000/v1/models
```

If Qwen is still unstable, the app can run with:

```
VLM_BACKEND=heuristic
```

Terminal 2 - Start locate-anything.cpp Wrapper

Start the wrapper around the locate-anything.cpp CLI:

```
cd /workspace/AMD_Hackathon
python scripts/locate_anything_cpp_server.py \
  --bin /workspace/locate-anything.cpp/build/examples/cli/locate-anything-cli \
  --model /workspace/locate-anything.cpp/models/locate-anything-q8_0.gguf \
  --mode fast \
  --host 127.0.0.1 \
  --port 8188
```

Health check:

```
curl http://127.0.0.1:8188/health
```

The wrapper caches repeated detections by model, mode, image path, image mtime, and prompt. This makes repeated app and cache runs faster.

Terminal 3 - Benchmark First

Run the benchmark before caching:

```
cd /workspace/AMD_Hackathon
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/benchmark_pipeline.py --frame-count 10
```

Outputs:

```
outputs/benchmark_pipeline.json
outputs/benchmark_pipeline.csv
```

The benchmark reports:

- `locator_cold_seconds`: locate-anything.cpp recomputed for the frame
- `locator_cached_seconds`: repeated wrapper call using cache when available
- `pipeline_seconds`: full app pipeline using normal cache behavior
- `pipeline_non_locator_seconds`: estimated non-locator pipeline time
- `estimated_cold_e2e_seconds`: cold locator plus non-locator pipeline estimate

Latest measured sample from AMD run:

```
Average locator latency: 4.837s
Average pipeline latency: 2.823s
```

Interpretation:

- The locator is the current bottleneck on cold frames.

- Pipeline time can be lower than locator time when the wrapper cache is warm.
- For jury numbers, report both cold locator and warm/cached demo latency.

To measure a fully warm run:

```
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/benchmark_pipeline.py --frame-count 10 --use-existing-locator-cache
```

Monitor AMD GPU Use

In another terminal:

```
watch -n 1 rocm-smi
```

For one-time detailed info:

```
rocm-smi
```

For locate-anything.cpp build validation:

```
grep -R "GGML_HIP" /workspace/locate-anything.cpp/build/CMakeCache.txt
grep -R "LA_GGML" /workspace/locate-anything.cpp/build/CMakeCache.txt
ldd /workspace/locate-anything.cpp/build/examples/cli/locate-anything-cli | grep -iE "hip|rocm"
```

Expected HIP-related build flags include:

```
GGML_HIP:BOOL=ON
GGML_HIP_GRAPHPS:BOOL=ON
GGML_HIP_MMQ_MFMA:BOOL=ON
```

If `rocm-smi` shows high VRAM but `GPU%` stays near zero during detection, the binary may still be mostly CPU-bound or only loading memory without meaningful GPU compute.

Build Or Resume Cached Stream

Build the Qwen plus locator cache:

```
cd /workspace/AMD_Hackathon
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/cache_qwen_outputs.py --frame-count 621
```

The cache script saves after every frame and resumes automatically. If it stops, run the same command again and it will skip completed frames.

To recompute everything:

```
VLM_BACKEND=qwen \  
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \  
LOCATOR_BACKEND=locate_anything_cpp_http \  
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \  
python scripts/cache_qwen_outputs.py --frame-count 621 --force
```

Cache file:

cache/qwen_stream.json

Cached records include:

- Qwen scene output
- risk/action/mode/target speed
- navigation desired path and floor region
- locate-anything.cpp bounding boxes
- path and floor source

Start Gradio Demo

Start the full demo with public sharing:

```
cd /workspace/AMD_Hackathon  
DEMO_FRAME_COUNT=621 \  
VLM_BACKEND=qwen \  
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \  
LOCATOR_BACKEND=locate_anything_cpp_http \  
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \  
GRADIO_SHARE=1 \  
./start.sh
```

Expected output:

Running on local URL: <http://0.0.0.0:7860>

Running on public URL: <https://...gradio.live>

If Qwen is unavailable, run the stable fallback:

```
DEMO_FRAME_COUNT=621 \  
VLM_BACKEND=heuristic \  
LOCATOR_BACKEND=locate_anything_cpp_http \  
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \  
GRADIO_SHARE=1 \  
./start.sh
```

If locate-anything.cpp is unavailable, run the safest fallback:

```
DEMO_FRAME_COUNT=621 \  
VLM_BACKEND=heuristic \  
LOCATOR_BACKEND=labels \  
./start.sh
```

```
GRADIO_SHARE=1 \  
./start.sh
```

Complete Command Set

Use this section as the single copy-paste reference during the demo.

1. Qwen VLM Server

```
cd /workspace/AMD_Hackathon  
./scripts/fix_aml_vllm_env.sh  
./scripts/start_qwen_vllm.sh  
  
curl http://localhost:8000/v1/models
```

2. locate-anything.cpp HTTP Wrapper

```
cd /workspace/AMD_Hackathon  
python scripts/locate_anything_cpp_server.py \  
    --bin /workspace/locate-anything.cpp/build/examples/cli/locate-anything-cli \  
    --model /workspace/locate-anything.cpp/models/locate-anything-q8_0.gguf \  
    --mode fast \  
    --host 127.0.0.1 \  
    --port 8188  
  
curl http://127.0.0.1:8188/health
```

3. Benchmark

```
cd /workspace/AMD_Hackathon  
VLM_BACKEND=qwen \  
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \  
LOCATOR_BACKEND=locate_anything_cpp_http \  
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \  
python scripts/benchmark_pipeline.py --frame-count 10
```

4. Warm Benchmark

```
cd /workspace/AMD_Hackathon  
VLM_BACKEND=qwen \  
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \  
LOCATOR_BACKEND=locate_anything_cpp_http \  
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \  
python scripts/benchmark_pipeline.py --frame-count 10 --use-existing-locator-cache
```

5. GPU Monitoring

```
watch -n 1 rocm-smi
```

```
grep -R "GGML_HIP" /workspace/locate-anything.cpp/build/CMakeCache.txt
grep -R "LA_GGML" /workspace/locate-anything.cpp/build/CMakeCache.txt
ldd /workspace/locate-anything.cpp/build/examples/cli/locate-anything-cli | grep -iE "hip|ro
```

6. Build Or Resume Cache

```
cd /workspace/AMD_Hackathon
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/cache_qwen_outputs.py --frame-count 621
```

7. Rebuild Cache From Scratch

```
cd /workspace/AMD_Hackathon
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/cache_qwen_outputs.py --frame-count 621 --force
```

8. Start Full Demo

```
cd /workspace/AMD_Hackathon
DEMO_FRAME_COUNT=621 \
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
GRADIO_SHARE=1 \
./start.sh
```

9. Start Stable Fallback Demo

```
cd /workspace/AMD_Hackathon
DEMO_FRAME_COUNT=621 \
VLM_BACKEND=heuristic \
LOCATOR_BACKEND=labels \
GRADIO_SHARE=1 \
./start.sh
```

10. Export This Runbook To PDF

```
cd /workspace/AMD_Hackathon
pandoc docs/COMMITTEE_RUNBOOK.md -o docs/COMMITTEE_RUNBOOK.pdf --pdf-engine=xelatex
```

Jury Focus Points

- Multimodal safety reasoning: RGB, thermal, radar, VLM, and locator outputs are fused into one robot command.
- Robot path is grounded to the floor region instead of floating over the image.
- Path color encodes target speed.
- STOP actions render as a point from the robot perspective, not a misleading end line.
- Qwen can provide scene and path estimates; local heuristics keep the demo robust if the VLM endpoint fails.
- `locate-anything.cpp` boxes are shown in the stream and in the detailed frame pane.
- Caching makes the long stream practical for a live demo while retaining model output per frame.

Operational Notes

Qwen Dependency Alignment

Run:

```
./scripts/fix_aml_vllm_env.sh
```

This aligns compatible versions for `huggingface-hub`, `fastapi`, `starlette`, and `prometheus-fastapi-instrumentator`.

Qwen Server Restart

Run:

```
./scripts/fix_aml_vllm_env.sh
./scripts/start_qwen_vllm.sh
```

Missing Cached Frame Output

Run or resume:

```
VLM_BACKEND=qwen \
QWEN_VLM_BASE_URL=http://localhost:8000/v1 \
LOCATOR_BACKEND=locate_anything_cpp_http \
LOCATE_ANYTHING_CPP_URL=http://127.0.0.1:8188/detect \
python scripts/cache_qwen_outputs.py --frame-count 621
```

Full Stream Frame Count

Start with:

```
DEMO_FRAME_COUNT=621 ./start.sh
```

locate-anything.cpp Runtime Check

Check:

```
watch -n 1 rocm-smi
grep -R "GGML_HIP" /workspace/locate-anything.cpp/build/CMakeCache.txt
```

Use wrapper cache for demo runs:

```
LOCATOR_BACKEND=locate_anything_cpp_http
```

For fastest uncached throughput, the next engineering step is replacing the CLI wrapper with a native long-running C/C++ server that keeps the GGUF model loaded. `## Source Code Appendix`

This section contains the actual source code for the main scripts used to run, benchmark, cache, and serve the demo.

start.sh

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
cd "$ROOT_DIR"

if [ -f ".env" ]; then
    set -a
    # shellcheck disable=SC1091
    source ".env"
    set +a
fi

detect_venv_dir() {
    if [ -n "${VIRTUAL_ENV:-}" ]; then
        printf '%s\n' "$VIRTUAL_ENV"
    elif [ -n "${VENV_DIR:-}" ]; then
        printf '%s\n' "$VENV_DIR"
    elif [ -d "venv" ]; then
        printf '%s\n' "venv"
    else
        printf '%s\n' ".venv"
    fi
}

VENV_DIR="$(detect_venv_dir)"

if [ -z "${VIRTUAL_ENV:-}" ]; then
    if [ ! -d "$VENV_DIR" ]; then
```



```

        echo "Virtual environment not found. Run ./install.sh first."
        exit 1
    fi

    # shellcheck disable=SC1091
    source "$VENV_DIR/bin/activate"
fi

if [ -z "${VLM_BACKEND:-}" ]; then
    if python - <<'PY' >/dev/null 2>&1
from urllib.request import urlopen
urlopen("http://localhost:8000/v1/models", timeout=1)
PY
    then
        export VLM_BACKEND=qwen
        export QWEN_VLM_BASE_URL="${QWEN_VLM_BASE_URL:-http://localhost:8000/v1}"
    else
        export VLM_BACKEND=heuristic
    fi
fi

export LOCATOR_BACKEND="${LOCATOR_BACKEND:-labels}"

GRADIO_SERVER_NAME="${GRADIO_SERVER_NAME:-0.0.0.0}"
GRADIO_SERVER_PORT="$(
    python - <<'PY'
import os
import socket

start_port = int(os.getenv("GRADIO_SERVER_PORT", "7860"))
host = "127.0.0.1"

for port in range(start_port, start_port + 100):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        try:
            sock.bind((host, port))
        except OSError:
            continue
        print(port)
        break
else:
    raise SystemExit(f"No free port found from {start_port} to {start_port + 99}")
PY
)"

export GRADIO_SERVER_NAME

```

```

export GRADIO_SERVER_PORT

echo "VLM_BACKEND=${VLM_BACKEND}"
echo "LOCATOR_BACKEND=${LOCATOR_BACKEND}"
echo "Starting Industrial Robot Copilot at http://localhost:${GRADIO_SERVER_PORT}"
exec python app.py

scripts/fix_amd_vllm_env.sh

#!/usr/bin/env bash
set -euo pipefail

python3 -m pip install --force-reinstall \
    "huggingface-hub==0.36.0" \
    "fastapi==0.115.14" \
    "starlette==0.46.2" \
    "prometheus-fastapi-instrumentator==7.1.0" \
    --break-system-packages

python3 scripts/check_vllm_env.py

scripts/start_qwen_vllm.sh

#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

detect_venv_dir() {
    if [ -n "${VIRTUAL_ENV:-}" ]; then
        printf '%s\n' "$VIRTUAL_ENV"
    elif [ -n "${VENV_DIR:-}" ]; then
        printf '%s\n' "$VENV_DIR"
    elif [ -d "venv" ]; then
        printf '%s\n' "venv"
    else
        printf '%s\n' ".venv"
    fi
}

VENV_DIR="$(detect_venv_dir)"

if [ -z "${VIRTUAL_ENV:-}" ] && [ -d "$VENV_DIR" ]; then
    # shellcheck disable=SC1091
    source "$VENV_DIR/bin/activate"

```

```

fi

if [ "${QWEN_VLLM_SKIP_ENV_CHECK:-0}" != "1" ]; then
    python3 scripts/check_vllm_env.py
fi

unset CUDA_VISIBLE_DEVICES
unset VLLM_USE_V1

export HIP_VISIBLE_DEVICES="${HIP_VISIBLE_DEVICES:-0}"
export ROCR_VISIBLE_DEVICES="${ROCR_VISIBLE_DEVICES:-0}"
export VLLM_TARGET_DEVICE="${VLLM_TARGET_DEVICE:-rocm}"

MODEL="${QWEN_VLM_MODEL:-Qwen/Qwen2.5-VL-7B-Instruct}"
PORT="${QWEN_VLLM_PORT:-8000}"

exec vllm serve "$MODEL" \
    --host 0.0.0.0 \
    --port "$PORT" \
    --trust-remote-code \
    --max-model-len "${QWEN_VLLM_MAX_MODEL_LEN:-8192}" \
    --gpu-memory-utilization "${QWEN_VLLM_GPU_MEMORY_UTILIZATION:-0.85}" \
    --max-num-seqs "${QWEN_VLLM_MAX_NUM_SEQS:-8}"

scripts/check_vllm_env.py

from importlib import metadata
import sys

def version_tuple(version):

    parts = []

    for part in version.split("."):
        number = ""

        for char in part:
            if char.isdigit():
                number += char
            else:
                break

        if not number:
            break

```

```

        parts.append(
            int(
                number
            )
        )

    return tuple(
        parts
    )

def package_version(name):

    try:
        return metadata.version(
            name
        )
    except metadata.PackageNotFoundError:
        return None

def fail(message):

    print(
        message,
        file=sys.stderr
    )
    raise SystemExit(
        1
    )

def main():

    hub = package_version(
        "huggingface-hub"
    )

    if hub is not None:
        hub_version = version_tuple(
            hub
        )

        if hub_version < (0, 34) or hub_version >= (1, 0):
            fail(
                "\n".join([

```

```

        "Bad vLLM dependency: transformers requires "
        "huggingface-hub>=0.34.0,<1.0.",
        f"Installed huggingface-hub: {hub}",
        "Fix:",
        "  python3 -m pip install --force-reinstall "
        "\"huggingface-hub==0.36.0\" --break-system-packages",
        "Or run:",
        "  ./scripts/fix_amd_vllm_env.sh"
    ])
)

fastapi = package_version(
    "fastapi"
)
starlette = package_version(
    "starlette"
)
instrumentator = package_version(
    "prometheus-fastapi-instrumentator"
)

if fastapi and version_tuple(fastapi) >= (0, 116):
    fail(
        "\n".join([
            "Bad vLLM web dependency: FastAPI >=0.116 can trigger "
            "prometheus_fastapi_instrumentator route crashes in this "
            "AMD image.",
            f"Installed fastapi: {fastapi}",
            f"Installed starlette: {starlette or 'missing'}",
            f"Installed prometheus-fastapi-instrumentator: "
            f"{instrumentator or 'missing'}",
            "Fix:",
            "  python3 -m pip install --force-reinstall "
            "\"fastapi==0.115.14\" \"starlette==0.46.2\" "
            "\"prometheus-fastapi-instrumentator==7.1.0\" "
            "--break-system-packages",
            "Or run:",
            "  ./scripts/fix_amd_vllm_env.sh"
        ])
    )

print(
    "vLLM dependency preflight OK"
)

```

```

if __name__ == "__main__":
    main()

scripts/locate_anything_cpp_server.py

import argparse
from http.server import BaseHTTPRequestHandler
from http.server import ThreadingHTTPServer
import hashlib
import json
from pathlib import Path
import subprocess
import tempfile
import time

def cache_key(model, mode, image_path, prompt):

    image = Path(
        image_path
    ).resolve()
    stat = image.stat()
    digest = hashlib.sha256()

    for value in [
        str(
            Path(
                model
            ).resolve()
        ),
        mode,
        str(
            image
        ),
        str(
            stat.st_size
        ),
        str(
            stat.st_mtime_ns
        ),
        prompt
    ]:
        digest.update(
            value.encode(
                "utf-8"
            )

```

```

        )
        digest.update(
            b"\0"
        )

    return digest.hexdigest()

def run_cli(config, image_path, prompt):

    with tempfile.NamedTemporaryFile(
        suffix=".json",
        delete=False
    ) as output_file:
        output_path = Path(
            output_file.name
        )

    cmd = [
        config["binary"],
        "detect",
        "--model",
        config["model"],
        "--input",
        image_path,
        "--prompt",
        prompt,
        "--mode",
        config["mode"],
        "--output",
        str(
            output_path
        )
    ]

    if config.get(
        "threads"
    ):
        cmd.extend([
            "--threads",
            str(
                config["threads"]
            )
        ])

    try:

```

```

start = time.perf_counter()
result = subprocess.run(
    cmd,
    check=False,
    capture_output=True,
    text=True
)
elapsed = time.perf_counter() - start

if result.returncode != 0:
    raise RuntimeError(
        result.stderr.strip()
        or result.stdout.strip()
        or f"{config['binary']} exited with {result.returncode}"
    )

payload = json.loads(
    output_path.read_text()
)
payload["_wrapper"] = {
    "cached":
        False,
    "seconds":
        elapsed,
    "mode":
        config["mode"]
}

return payload
finally:
    output_path.unlink(
        missing_ok=True
    )

```

```

class LocateAnythingHandler(BaseHTTPRequestHandler):

    server_version = "LocateAnythingCppWrapper/1.0"

    def do_GET(self):

        if self.path != "/health":
            self.send_error(
                404
            )
            return

```



```

self.write_json({
    "ok":
        True,
    "mode":
        self.server.config["mode"],
    "model":
        self.server.config["model"]
})

def do_POST(self):

    if self.path != "/detect":
        self.send_error(
            404
        )
        return

    try:
        payload = self.read_json()
        image_path = str(
            payload["image_path"]
        )
        prompt = str(
            payload["prompt"]
        )
        bypass_cache = bool(
            payload.get(
                "bypass_cache",
                False
            )
        )

        if not Path(
            image_path
        ).exists():
            raise RuntimeError(
                f"image_path does not exist: {image_path}"
            )

        key = cache_key(
            self.server.config["model"],
            self.server.config["mode"],
            image_path,
            prompt
        )

```

```

cache_path = self.server.cache_dir / f"{key}.json"

if cache_path.exists() and not bypass_cache:
    cached = json.loads(
        cache_path.read_text()
    )
    cached.setdefault(
        "_wrapper",
        {}
    )
    cached["_wrapper"]["cached"] = True
    self.write_json(
        cached
    )
    return

result = run_cli(
    self.server.config,
    image_path,
    prompt
)
result.setdefault(
    "_wrapper",
    {}
)
result["_wrapper"]["bypass_cache"] = bypass_cache
cache_path.write_text(
    json.dumps(
        result,
        indent=2
    )
)
self.write_json(
    result
)
except Exception as exc:
    self.send_response(
        500
    )
    self.send_header(
        "Content-Type",
        "application/json"
    )
    self.end_headers()
    self.wfile.write(
        json.dumps({

```

```

        "error":
            f"{type(exc).__name__}: {exc}"
    }).encode(
        "utf-8"
    )
)

def read_json(self):

    length = int(
        self.headers.get(
            "Content-Length",
            "0"
        )
    )

    return json.loads(
        self.rfile.read(
            length
        ).decode(
            "utf-8"
        )
    )

def write_json(self, payload):

    data = json.dumps(
        payload
    ).encode(
        "utf-8"
    )
    self.send_response(
        200
    )
    self.send_header(
        "Content-Type",
        "application/json"
    )
    self.send_header(
        "Content-Length",
        str(
            len(
                data
            )
        )
    )
)

```

```

        self.end_headers()
        self.wfile.write(
            data
        )

def log_message(self, fmt, *args):

    print(
        f"{self.address_string()} - {fmt % args}",
        flush=True
    )

def main():

    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--host",
        default="127.0.0.1"
    )
    parser.add_argument(
        "--port",
        type=int,
        default=8188
    )
    parser.add_argument(
        "--bin",
        required=True
    )
    parser.add_argument(
        "--model",
        required=True
    )
    parser.add_argument(
        "--mode",
        default="fast"
    )
    parser.add_argument(
        "--threads"
    )
    parser.add_argument(
        "--cache-dir",
        default="cache/locate_anything_cpp"
    )
    args = parser.parse_args()

```

```

cache_dir = Path(
    args.cache_dir
)
cache_dir.mkdir(
    parents=True,
    exist_ok=True
)

server = ThreadingHTTPServer(
    (
        args.host,
        args.port
    ),
    LocateAnythingHandler
)
server.config = {
    "binary":
        args.bin,
    "model":
        args.model,
    "mode":
        args.mode,
    "threads":
        args.threads
}
server.cache_dir = cache_dir

print(
    f"locate-anything.cpp wrapper listening on http://{args.host}:{args.port}",
    flush=True
)
print(
    f"mode={args.mode} cache_dir={cache_dir}",
    flush=True
)
server.serve_forever()

if __name__ == "__main__":
    main()

```

scripts/benchmark_pipeline.py

```

import argparse
import csv
import json

```

```

import os
from pathlib import Path
import shutil
import subprocess
import sys
import time
from contextlib import contextmanager

ROOT_DIR = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT_DIR))

from src.loader import load_frame
from src.locate_anything import locate_objects
from src.pipeline import run_pipeline

def frame_ids(limit):

    frames = [
        line.strip()
        for line in Path("data/industrial_subset/frames.txt").read_text().splitlines()
        if line.strip()
    ]

    if limit and limit > 0:
        return frames[:limit]

    return frames

def rocm_smi_snapshot():

    if shutil.which(
        "rocm-smi"
    ) is None:
        return {
            "available":
                False
        }

    try:
        result = subprocess.run(
            [
                "rocm-smi",
                "--showuse",
                "--showmemuse",
            ]

```

```

        "--showmeminfo",
        "vram",
        "--json"
    ],
    check=False,
    capture_output=True,
    text=True,
    timeout=10
)
except Exception as exc:
    return {
        "available":
            False,
        "error":
            f"{type(exc).__name__}: {exc}"
    }

if result.returncode != 0:
    return {
        "available":
            False,
        "error":
            result.stderr.strip()
    }

try:
    return {
        "available":
            True,
        "raw":
            json.loads(
                result.stdout
            )
    }
except json.JSONDecodeError:
    return {
        "available":
            True,
        "raw_text":
            result.stdout
    }

def summarize_gpu(snapshot):

    if not snapshot.get(

```

```

        "available"
    ):
        return {}

    raw = snapshot.get(
        "raw",
        {}
    )

    summary = {}

    for gpu, values in raw.items():
        if not isinstance(
            values,
            dict
        ):
            continue

        summary[gpu] = {
            key:
                value
            for key, value in values.items()
            if any(
                token in key.lower()
                for token in [
                    "use",
                    "memory",
                    "vram"
                ]
            )
        }

    return summary

def timed_call(fn):

    start = time.perf_counter()
    value = fn()
    elapsed = time.perf_counter() - start

    return value, elapsed

@contextmanager
def temporary_env(updates):

```



```

previous = {
    key:
        os.environ.get(
            key
        )
    for key in updates
}

try:
    for key, value in updates.items():
        if value is None:
            os.environ.pop(
                key,
                None
            )
        else:
            os.environ[key] = str(
                value
            )

    yield
finally:
    for key, value in previous.items():
        if value is None:
            os.environ.pop(
                key,
                None
            )
        else:
            os.environ[key] = value

def benchmark_frame(frame_id, bypass_locator_cache):

    frame = load_frame(
        frame_id
    )

    gpu_before = rocm_smi_snapshot()

    with temporary_env({
        "LOCATE_ANYTHING_CPP_BYPASS_CACHE":
            "1" if bypass_locator_cache else None
    }):
        located, locator_cold_seconds = timed_call(

```

```

        lambda: locate_objects(
            frame
        )
    )

    with temporary_env({
        "LOCATE_ANYTHING_CPP_BYPASS_CACHE":
            None
    }):
        cached_located, locator_cached_seconds = timed_call(
            lambda: locate_objects(
                frame
            )
        )

    result, pipeline_seconds = timed_call(
        lambda: run_pipeline(
            frame_id
        )
    )

    gpu_after = rocm_smi_snapshot()

    scene = result["scene"]
    decision = result["decision"]
    plan = result["plan"]

    return {
        "frame_id":
            frame_id,
        "locator_backend":
            os.getenv(
                "LOCATOR_BACKEND",
                "labels"
            ),
        "vlm_backend":
            os.getenv(
                "VLM_BACKEND",
                "heuristic"
            ),
        "scene_source":
            scene.get(
                "scene_source",
                "unknown"
            ),
        "risk_level":

```

```

        decision["risk_level"],
    "action":
        plan["action"],
    "target_speed":
        plan["target_speed"],
    "locator_seconds":
        round(
            locator_cold_seconds,
            4
        ),
    "locator_cold_seconds":
        round(
            locator_cold_seconds,
            4
        ),
    "locator_cached_seconds":
        round(
            locator_cached_seconds,
            4
        ),
    "pipeline_non_locator_seconds":
        round(
            max(
                0.0,
                pipeline_seconds - locator_cached_seconds
            ),
            4
        ),
    "estimated_cold_e2e_seconds":
        round(
            locator_cold_seconds
            + max(
                0.0,
                pipeline_seconds - locator_cached_seconds
            ),
            4
        ),
    "pipeline_seconds":
        round(
            pipeline_seconds,
            4
        ),
    "located_count":
        len(
            located
        ),

```

```

"cached_located_count":
    len(
        cached_located
    ),
"pipeline_located_count":
    len(
        scene.get(
            "located_objects",
            []
        )
    ),
"path_source":
    plan["navigation"].get(
        "desired_path_source"
    ),
"floor_source":
    plan["navigation"].get(
        "floor_region_source"
    ),
"gpu_before":
    summarize_gpu(
        gpu_before
    ),
"gpu_after":
    summarize_gpu(
        gpu_after
    )
}

```

```

def write_csv(path, records):

```

```

    if not records:
        return

```

```

    fieldnames = [
        "frame_id",
        "locator_backend",
        "vlm_backend",
        "scene_source",
        "risk_level",
        "action",
        "target_speed",
        "locator_seconds",
        "locator_cold_seconds",
        "locator_cached_seconds",
    ]

```

```

        "pipeline_non_locator_seconds",
        "estimated_cold_e2e_seconds",
        "pipeline_seconds",
        "located_count",
        "cached_located_count",
        "pipeline_located_count",
        "path_source",
        "floor_source"
    ]

    with path.open(
        "w",
        newline=""
    ) as file:
        writer = csv.DictWriter(
            file,
            fieldnames=fieldnames
        )
        writer.writeheader()

        for record in records:
            writer.writerow({
                key:
                    record.get(
                        key
                    )
                for key in fieldnames
            })

def main():

    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--frame-count",
        type=int,
        default=5
    )
    parser.add_argument(
        "--output-json",
        default="outputs/benchmark_pipeline.json"
    )
    parser.add_argument(
        "--output-csv",
        default="outputs/benchmark_pipeline.csv"
    )

```

```

parser.add_argument(
    "--use-existing-locator-cache",
    action="store_true",
    help=(
        "Do not force the locate-anything.cpp HTTP wrapper to recompute "
        "the first locator call for each frame."
    )
)
args = parser.parse_args()

records = []

for idx, frame_id in enumerate(
    frame_ids(
        args.frame_count
    ),
    start=1
):
    print(
        f"[{idx}/{args.frame_count}] benchmarking {frame_id}",
        flush=True
    )
    record = benchmark_frame(
        frame_id,
        not args.use_existing_locator_cache
    )
    records.append(
        record
    )
    print(
        json.dumps(
            {
                "frame_id":
                    record["frame_id"],
                "locator_seconds":
                    record["locator_seconds"],
                "locator_cached_seconds":
                    record["locator_cached_seconds"],
                "pipeline_seconds":
                    record["pipeline_seconds"],
                "estimated_cold_e2e_seconds":
                    record["estimated_cold_e2e_seconds"],
                "located_count":
                    record["located_count"]
            }
        ),

```

```

        flush=True
    )

    output_json = Path(
        args.output_json
    )
    output_csv = Path(
        args.output_csv
    )
    output_json.parent.mkdir(
        parents=True,
        exist_ok=True
    )
    output_csv.parent.mkdir(
        parents=True,
        exist_ok=True
    )
    output_json.write_text(
        json.dumps(
            {
                "environment":
                    {
                        key:
                            os.getenv(
                                key,
                                ""
                            )
                        for key in [
                            "VLM_BACKEND",
                            "QWEN_VLM_BASE_URL",
                            "QWEN_VLM_MODEL",
                            "LOCATOR_BACKEND",
                            "LOCATE_ANYTHING_CPP_BIN",
                            "LOCATE_ANYTHING_CPP_MODEL",
                            "LOCATE_ANYTHING_CPP_MODE"
                        ]
                    },
                "records":
                    records
            },
            indent=2
        )
    )
    write_csv(
        output_csv,
        records
    )

```

```

    )

    if records:
        locator_avg = sum(
            record["locator_seconds"]
            for record in records
        ) / len(records)
        pipeline_avg = sum(
            record["pipeline_seconds"]
            for record in records
        ) / len(records)
        cached_locator_avg = sum(
            record["locator_cached_seconds"]
            for record in records
        ) / len(records)
        cold_e2e_avg = sum(
            record["estimated_cold_e2e_seconds"]
            for record in records
        ) / len(records)
        print(
            f"Average locator latency: {locator_avg:.3f}s"
        )
        print(
            f"Average cached locator latency: {cached_locator_avg:.3f}s"
        )
        print(
            f"Average pipeline latency: {pipeline_avg:.3f}s"
        )
        print(
            f"Average estimated cold end-to-end latency: {cold_e2e_avg:.3f}s"
        )

    print(
        f"Wrote {output_json} and {output_csv}"
    )

if __name__ == "__main__":
    main()

scripts/cache_qwen_outputs.py

import argparse
import json
import os
from pathlib import Path

```



```

import sys
import time

ROOT_DIR = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT_DIR))

from src.pipeline import run_pipeline

def frame_ids(limit):

    frames = [
        line.strip()
        for line in Path("data/industrial_subset/frames.txt").read_text().splitlines()
        if line.strip()
    ]

    if limit and limit > 0:
        return frames[:limit]

    return frames

def compact(frame_id, result):

    world = result["world"]
    decision = result["decision"]
    plan = result["plan"]
    scene = result["scene"]

    human = world["human"]
    distance = human["distance"]

    return {
        "frame_id":
            frame_id,
        "scene_source":
            scene.get(
                "scene_source",
                "unknown"
            ),
        "risk_level":
            decision["risk_level"],
        "action":
            plan["action"],
        "mode":

```

```

        plan["mode"],
        "target_speed":
            plan["target_speed"],
        "human_present":
            human["present"],
        "human_distance":
            distance,
        "environment_type":
            scene.get(
                "environment_type",
                "unknown"
            ),
        "hazards":
            scene.get(
                "hazards",
                []
            ),
        "located_count":
            len(
                scene.get(
                    "located_objects",
                    []
                )
            ),
        "located_objects":
            scene.get(
                "located_objects",
                []
            ),
        "navigation":
            plan.get(
                "navigation",
                {}
            ),
        "reasoning":
            decision["reasoning"],
    }

```

```

def load_existing_records(output_path):

    if not output_path.exists():
        return []

    try:
        payload = json.loads(

```

```

        output_path.read_text()
    )
except json.JSONDecodeError:
    return []

return payload.get(
    "records",
    []
)

def write_records(output_path, records):

    temp_path = output_path.with_suffix(
        output_path.suffix + ".tmp"
    )
    temp_path.write_text(
        json.dumps(
            {
                "records":
                    records
            },
            indent=2
        )
    )
    temp_path.replace(
        output_path
    )

def main():

    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--frame-count",
        type=int,
        default=int(
            os.getenv(
                "QWEN_CACHE_FRAME_COUNT",
                "50"
            )
        )
    )
    parser.add_argument(
        "--output",
        default=os.getenv(

```

```

        "QWEN_STREAM_CACHE",
        "cache/qwen_stream.json"
    )
)
parser.add_argument(
    "--force",
    action="store_true",
    help="Recompute frames even if they already exist in the cache."
)
args = parser.parse_args()

os.environ.setdefault(
    "VLM_BACKEND",
    "qwen"
)
os.environ.setdefault(
    "LOCATOR_BACKEND",
    "labels"
)

output_path = Path(
    args.output
)
output_path.parent.mkdir(
    parents=True,
    exist_ok=True
)

records = (
    []
    if args.force
    else load_existing_records(
        output_path
    )
)

existing = {
    record.get(
        "frame_id"
    )
    for record in records
}

frames = frame_ids(
    args.frame_count
)

```

```

for idx, frame_id in enumerate(
    frames,
    start=1
):

    if frame_id in existing:
        print(
            f"[{idx}/{len(frames)}] skip {frame_id} already cached",
            flush=True
        )
        continue

    print(
        f"[{idx}/{len(frames)}] caching {frame_id}",
        flush=True
    )
    start = time.perf_counter()
    result = run_pipeline(
        frame_id
    )
    records.append(
        compact(
            frame_id,
            result
        )
    )
    existing.add(
        frame_id
    )
    write_records(
        output_path,
        records
    )
    elapsed = time.perf_counter() - start
    print(
        f"[{idx}/{len(frames)}] wrote {frame_id} in {elapsed:.2f}s",
        flush=True
    )

    print(
        f"Wrote {len(records)} records to {output_path}"
    )

if __name__ == "__main__":
    main()

```